Medição Empírica de Dependabilidade do Apache Cassandra sob Injeção de Faltas: o Impacto dos Níveis de Consistência

Caio Farias Cavalcanti[1] e Gustavo Rau de Almeida Callou[1]

Universidade Federal Rural de Pernambuco (UFRPE), Recife–PE, Brasil
{caio.farias,gustavo.callou}@ufrpe.br

Resumo Sistemas de banco de dados distribuídos como o Apache Cassandra oferecem níveis de consistência ajustáveis que regulam o compromisso entre disponibilidade e consistência. Embora bem compreendido em teoria, esse compromisso carece de medições empíricas sobre como diferentes níveis de consistência afetam a dependabilidade do serviço sob faltas reais. Este trabalho apresenta um estudo de medição que injeta cinco tipos de falta (crash de réplica, crash de coordenador, atraso de rede, perda de pacotes e partição de rede) no Cassandra sob três níveis de consistência (ONE, QUORUM, ALL) e três níveis de carga, com três repetições cada (135 execuções). Os resultados quantificam dois fenômenos distintos: faltas que geram indisponibilidade (crash e partição), em que ONE e QUORUM mascaram completamente a falta enquanto ALL apresenta taxa de erro de até 25%; e faltas de temporização (atraso e perda), em que a vazão degrada ∼98% independentemente do nível de consistência. Mede-se ainda o tempo de reparo (MTTR) por tipo de falta, de ∼15 s (crash) a ∼32 s (partição). As medições estendem trabalhos anteriores ao introduzir cenários de partição não estudados previamente e ao fornecer valores empíricos de disponibilidade observada de serviço e MTTR.

Keywords: Dependabilidade · Apache Cassandra · Injeção de faltas · Níveis de consistência · Disponibilidade · Tolerância a faltas.

1 Introdução

Bancos de dados NoSQL distribuídos tornaram-se infraestrutura central de aplicações de larga escala, oferecendo escalabilidade horizontal e tolerância a faltas por meio de replicação e consenso. O Apache Cassandra é um representante proeminente dessa categoria, adotado por organizações que demandam alta disponibilidade sobre grandes volumes de dados [1]. Uma característica definidora do Cassandra é a *consistência ajustável*: para cada operação, a aplicação escolhe quantas réplicas devem participar, regulando explicitamente o compromisso entre consistência e disponibilidade previsto pelo teorema CAP [2,3].

Esse compromisso é bem compreendido em nível teórico. O que permanece menos explorado é a *quantificação empírica* de como a escolha do nível de consistência afeta a dependabilidade do serviço entregue ao cliente quando o sistema

enfrenta faltas reais. Saber que ALL "prioriza consistência"não informa *quanto* da disponibilidade se perde sob uma falha concreta, nem *quão rápido* o serviço se recupera. São medidas de dependabilidade [4,5] de que operadores precisam para decidir configurações.

Este trabalho aborda essa lacuna por meio de um estudo de medição controlado: um cluster Cassandra de cinco nós em nuvem é submetido a cinco classes de falta sob três níveis de consistência e três cargas, registrando-se vazão, latência e taxa de erro antes, durante e após cada falta.

O objeto de estudo é o nível de consistência, e o Cassandra se presta a isolá-lo porque o nível é escolhido por operação: ONE, QUORUM e ALL podem ser comparados no *mesmo* cluster, com os mesmos dados e a mesma falta. Em sistemas baseados em consenso, como o etcd (Raft), a garantia é em grande parte fixada pelo protocolo. O método de injeção (`systemctl`, `tc/netem`, `iptables` e série temporal no cliente) não depende do Cassandra; se os compromissos observados se generalizam fica como trabalho futuro.

As contribuições são: (i) uma metodologia reproduzível de injeção de faltas para Cassandra, cobrindo crash, temporização, omissão e partição com controle preciso da linha do tempo, com *scripts* e dados publicamente disponíveis [14]; (ii) a quantificação empírica do impacto do nível de consistência sobre a dependabilidade, distinguindo faltas de disponibilidade de faltas de temporização; e (iii) valores medidos de disponibilidade observada e MTTR, incluindo cenários de partição não tratados na literatura correlata. O escopo é restrito à medição: esta é a etapa experimental de um projeto de graduação, tendo a modelagem analítica como etapa subsequente.

O estudo é guiado por três questões de pesquisa:

- **RQ1:** Como o nível de consistência do Cassandra afeta a disponibilidade do serviço sob faltas de *crash* e partição de rede?
- **RQ2:** Como o nível de consistência afeta a degradação de desempenho sob faltas de temporização de rede (atraso e perda de pacotes)?
- **RQ3:** Como diferentes tipos de falta afetam o tempo de recuperação do Cassandra (MTTR)?

A Seção 2 revisa os trabalhos relacionados e a Seção 3, os conceitos utilizados. A Seção 4 descreve o desenho experimental e a Seção 5, o aparato. A Seção 6 apresenta e discute os resultados, e a Seção 7 conclui.

2 Trabalhos Relacionados

A avaliação de dependabilidade do Cassandra tem sido abordada por medição e por modelagem analítica. Ferreira et al. [6] avaliaram experimentalmente o desempenho do Cassandra sob diferentes níveis de consistência e cargas, estabelecendo uma base de comparação de vazão e latência em operação normal, sem caracterizar o comportamento sob indisponibilidade de nós ou degradação de rede. Adotamos parâmetros de carga compatíveis e acrescentamos uma fase sistemática de injeção de faltas.

Outros estudos cobrem dimensões complementares. Rosselli et al. [10] avaliaram a disponibilidade e a tolerância a faltas do Cassandra medindo o impacto da queda de um nó sobre vazão e latência, sem porém variar sistematicamente o nível de consistência nem cobrir faltas de rede como atraso, perda e partição. Rahman et al. [11] propuseram um arcabouço, integrado ao YCSB, para medir anomalias de consistência em sistemas chave-valor, e o projeto Jepsen [12] submete bancos distribuídos, incluindo o Cassandra, a partições e crashes para verificar *violações de correção*; nosso trabalho, em vez disso, quantifica medidas de dependabilidade (taxa de erro, degradação de vazão e MTTR) em função do nível de consistência.

Gomes et al. [9] propuseram um modelo analítico de dependabilidade para o Cassandra baseado em redes de Petri estocásticas generalizadas (GSPN), estimando disponibilidade a partir de MTTF e MTTR adotados de forma hipotética (MTTF de 2880 h e 5760 h; MTTR de 0,5 h e 1 h), à falta de medições diretas; este trabalho fornece MTTR *medido empiricamente* por tipo de falta.

A injeção de faltas é técnica consolidada de *fault forecasting* para a avaliação experimental de dependabilidade [4,7,8]. Partições de rede, que materializam o teorema CAP, não foram, até onde sabemos, caracterizadas como medida de dependabilidade nos trabalhos correlatos sobre Cassandra.

3 Fundamentação Teórica

3.1 Dependabilidade e a cadeia falta–erro–falha

Dependabilidade é a capacidade de um sistema de entregar um serviço no qual se pode justificadamente confiar [4], englobando atributos como disponibilidade (prontidão para o serviço correto), confiabilidade (continuidade do serviço correto) e manutenibilidade (capacidade de reparo). O arcabouço de Avizienis et al. estrutura as ameaças à dependabilidade na cadeia causal *falta* $\rightarrow$ *erro* $\rightarrow$ *falha*: uma **falta** é a causa hipotética ou real de um erro (p. ex., um processo que termina); um **erro** é a parte do estado do sistema que pode levar a uma falha (p. ex., uma réplica inalcançável); e uma **falha** ocorre quando o serviço entregue desvia do correto (p. ex., uma operação que retorna erro ao cliente). Entre o erro e a falha atuam os *mecanismos de tolerância a faltas*: quando um mecanismo impede que um erro se propague até o serviço, diz-se que a falta foi *mascarada*; a injeção controlada de faltas mede se cada falta atravessa essas defesas.

3.2 Replicação e níveis de consistência do Cassandra

O Cassandra particiona os dados em um anel de *tokens* e replica cada partição em um número de nós dado pelo *fator de replicação* (RF). Com $RF = 3$, cada item existe em três nós distintos, o que constitui seu mecanismo fundamental de tolerância a faltas.

O *nível de consistência* (Consistency Level, CL) determina, por operação, quantas réplicas devem responder para que ela seja considerada bem-sucedida:

ONE exige uma réplica (tolera a perda de até $RF - 1$ réplicas); **QUORUM** exige a maioria, $\lfloor RF/2 \rfloor + 1$ réplicas (duas, para $RF = 3$; tolera a perda de uma); e **ALL** exige todas as RF réplicas (não tolera perda alguma).

Um nível de consistência é um parâmetro por operação, e não um *modelo* de consistência; o modelo resultante depende dos níveis usados nas leituras (R réplicas) e nas escritas (W). Com ONE, uma leitura pode retornar dado desatualizado, e o sistema oferece apenas *consistência eventual*. Quando $R + W > RF$, como com QUORUM em ambas ou com ALL, todo conjunto de leitura intersecta as réplicas que confirmaram a última escrita concluída. Isso ainda é mais fraco que a *linearizabilidade*, que no Cassandra exige *lightweight transactions* (Paxos), não usadas aqui. Medimos o *custo* em disponibilidade e desempenho de cada nível, e não a consistência obtida, objeto de [11,12].

O CL é, portanto, um mecanismo em nível de aplicação para controlar o comportamento observado durante falhas de nó e de rede: ONE favorece disponibilidade (responde com qualquer réplica alcançável), enquanto ALL recusa-se a responder sem todas. Um aspecto importante para interpretar os resultados é que, em Cassandra, *toda escrita é enviada a todas as réplicas* independentemente do CL, que define apenas quantas confirmações o coordenador aguarda.

3.3 Atributos de dependabilidade mensuráveis

A partir da observação do serviço sob faltas derivam-se as medidas de interesse, provenientes de duas tradições complementares: os atributos da taxonomia de Avizienis et al. [4] e a *performability* de Meyer [5]. A **disponibilidade** (prontidão para o serviço correto) é medida no nível operacional, pela fração de operações atendidas corretamente, isto é, $A = 1 - (\text{erros}/\text{total de operações})$; chamamos essa grandeza de *disponibilidade observada de serviço*. Ela se afasta da disponibilidade estacionária clássica $A = \text{MTTF}/(\text{MTTF} + \text{MTTR})$ por duas razões: as faltas são injetadas em instantes fixos, de modo que não há MTTF a medir; e, num sistema que atende requisições, cada operação é uma demanda de serviço, de modo que a fração atendida corretamente é a disponibilidade que o cliente experimenta, a forma baseada em requisições usada em objetivos de nível de serviço. Como depende do intervalo de observação, reportamo-la sobre a execução inteira e dentro da janela de falta. A **performability** [5] é o grau de degradação de vazão e latência durante a falta, quando o serviço permanece disponível mas com qualidade reduzida; não é um atributo da taxonomia de Avizienis et al., e sim uma medida composta de desempenho e dependabilidade, adequada a sistemas degradáveis. A **manutenibilidade** é avaliada pelo tempo de recuperação (MTTR) após a falta injetada.

4 Metodologia

4.1 Desenho experimental

O estudo organiza-se em duas fases. A *fase de baseline* mede vazão e latência em regime normal, variando o nível de consistência (ONE, QUORUM, ALL)

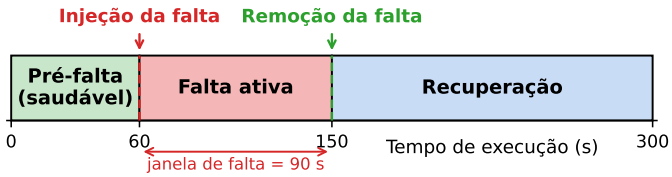


Figura 1. Linha do tempo controlada de uma execução de injeção de faltas.

e a carga (10, 30 e 50 *threads* de cliente), com três repetições por célula. A *fase de injeção de faltas* acrescenta a dimensão do tipo de falta, totalizando $5 \times 3 \times 3 \times 3 = 135$ execuções controladas.

4.2 Tipos de falta injetados

Cinco tipos de falta cobrem as principais classes da taxonomia de dependabilidade. **Crash de réplica** (`node_crash`): término do processo Cassandra em um nó réplica, via `systemctl stop`, que testa o mascaramento por replicação. Trata-se de um desligamento *ordenado*, em que o nó anuncia sua saída aos pares; a escolha isola a perda da réplica do atraso de detecção de falha e permite reinício limpo numa bateria automatizada longa (o término abrupto é discutido nas ameaças à validade). **Crash de coordenador** (`coord_crash`): término do processo no nó de contato do cliente, testando o *failover* de roteamento do *driver*. **Atraso de rede** (`net_delay`): inserção de 200 ms $\pm$ 50 ms de latência no tráfego de saída de um nó, via `tc/netem` (falta de temporização). **Perda de pacotes** (`net_loss`): descarte de 25% dos pacotes de saída de um nó, também via `tc/netem` (falta de omissão). **Partição de rede** (`net_partition`): isolamento de um nó em relação aos demais, via regras `iptables`, mantendo o nó vivo porém incomunicável, o que materializa o cenário do teorema CAP.

4.3 Linha do tempo de cada execução

Cada execução dura 300 s e segue uma linha do tempo controlada (Fig. 1): 60 s em regime saudável; injeção da falta em $t = 60$ s; falta ativa por 90 s; remoção em $t = 150$ s; e recuperação até $t = 300$ s. A janela de 90 s excede o tempo de detecção de falha do protocolo de *gossip* do Cassandra (da ordem de 10 s), garantindo uma janela de falta efetivamente detectada e longa o bastante para medição.

4.4 Carga de trabalho e métricas

A carga é gerada pelo YCSB (Yahoo! Cloud Serving Benchmark) [13] com o *Workload A* (50% leituras, 50% atualizações) sobre 100.000 registros, com execuções limitadas por tempo (300 s). O YCSB registra a cada 10 s a vazão, a latência e as contagens de operações bem-sucedidas e com erro; os instantes reais de injeção,



Figura 2. Arquitetura do ambiente experimental: cluster Cassandra 4.1 de cinco nós `e2-standard-2` (RF=3, `SimpleStrategy`) e cliente de carga YCSB (`e2-medium`), co-localizados na zona `us-central1-a` do Google Cloud Platform.

remoção e recuperação, registrados pelo injetor, alinham essa série às janelas pré, durante e pós-falta.

Dessas séries calculamos a **taxa de erro** (erros sobre total de operações) e a **disponibilidade observada de serviço** (1− taxa de erro), usadas para crash e partição; a **degradação de vazão**, $1 - \text{vazão}_{\text{durante}}/\text{vazão}_{\text{pré}}$, usada para atraso e perda; e o **MTTR**, o intervalo entre a remoção da falta e a confirmação, via `nodetool status`, de que o cluster voltou a cinco nós ativos. Por construção, o MTTR só é significativo para faltas que derrubam o nó do *gossip* (crash e partição); para faltas de temporização, a recuperação relevante é a da vazão, observável na janela pós-falta.

5 Ambiente e Aparato Experimental

5.1 Infraestrutura

O experimento foi executado no Google Cloud Platform, na zona `us-central1-a` (Fig. 2). O cluster Cassandra tem cinco máquinas virtuais `e2-standard-2` (2 vCPUs, 8 GB de RAM cada) executando Cassandra 4.1 sobre Ubuntu 22.04, e a carga é gerada por uma VM dedicada (`e2-medium`, 2 vCPUs, 4 GB) co-localizada na mesma zona, de modo que a latência cliente–cluster (∼0,3 ms) não seja fator de confusão.

5.2 Configuração do Cassandra

O cluster usa $RF = 3$ com `SimpleStrategy` (adequada a um único *datacenter*), `num_tokens`=256, `Murmur3Partitioner`, *snitch* `GossipingPropertyFileSnitch` e dois nós *seeds*. A distribuição de *tokens* resultante atribui a cada nó ∼58–62% do espaço de chaves em propriedade efetiva, o que é relevante para a interpretação dos resultados sob CL=ALL.

Tabela 1. Parâmetros do experimento.

Parâmetro	Valor	Parâmetro	Valor
Nós Cassandra	5 × e2-standard-2	Carga (threads)	10, 30, 50
Cliente de carga	1 × e2-medium	Repetições/célula	3
Versão do Cassandra	4.1	Carga de trabalho	YCSB A (50/50 leit./atual.)
Fator de replicação	3 (SimpleStrategy)	Registros	100.000
Níveis de consistência	ONE, QUORUM, ALL	Duração	300 s
Execuções (faltas)	135	Janela de falta	90 s ($t = 60$–150 s)

5.3 Carga e ferramentas

A geração de carga utiliza o YCSB 0.17.0 [13]. A injeção de faltas é realizada por *scripts* que orquestram, sincronizados com a execução do YCSB, o término e o reinício de processos (`systemctl`) para os crashes, a manipulação de disciplina de fila (`tc/netem`) para atraso e perda, e regras de *firewall* (`iptables`) para a partição. Verificação pós-remoção e limpeza idempotente impedem que estado residual de falta contamine execuções subsequentes, requisito crítico em uma bateria longa de execuções automatizadas. A Tabela 1 resume os parâmetros centrais. Os *scripts* de injeção e de análise, os logs brutos e os dados por execução estão publicamente disponíveis [14].

6 Estudo de Caso: Resultados e Discussão

Na fase de *baseline* sem faltas, o coeficiente de variação entre repetições ficou abaixo de 3%. Os resultados agrupam-se pelo atributo predominante: disponibilidade (crash, partição) e *performability* (faltas de temporização).

6.1 Disponibilidade sob crash e partição

A Tabela 2 apresenta a taxa de erro média por tipo de falta e nível de consistência. O padrão é nítido e consistente com a teoria do quórum: sob crash de réplica ou de coordenador, ONE e QUORUM mantêm taxa de erro praticamente nula (zero operações com falha em todas as células, exceto uma única em $2{,}6 \times 10^7$ sob ONE no crash de coordenador), pois com $RF = 3$ a perda de uma réplica deixa duas disponíveis, suficientes para ONE (uma) e QUORUM (duas). A falta é, portanto, *mascarada* pela replicação, e ambos entregam disponibilidade observada de serviço de ∼100%. Sob ALL, a exigência das três réplicas faz falhar toda operação cujo conjunto de réplicas inclua o nó indisponível, resultando em taxa de erro média de ∼24–25% sobre a execução completa, valor menor que a fração de réplicas afetadas porque é diluído pelos períodos saudáveis.

A diluição não é proporcional ao tempo (90 de 300 s, 30%). A vazão após a falta (∼4400–5700 ops/s) é menor que antes dela (∼7100–8200 ops/s), de modo que a janela de falta concentra 33–41% das operações da execução. Atribuímos a vazão pós-falta menor à reintegração do nó, que sob ALL participa de toda operação sobre suas réplicas; essa explicação é inferida do comportamento do Cassandra, e não instrumentada.

Tabela 2. Taxa de erro (%) por tipo de falta e nível de consistência (média ± desvio padrão sobre cargas e repetições; $n = 9$ por célula: 3 cargas × 3 repetições). Entre parênteses, o número absoluto de operações com falha somado sobre as nove execuções; cada célula totaliza 1,6–2,8 × 10^7 operações.

Tipo de falta	ONE	QUORUM	ALL
Crash de réplica	0,00±0,00 (0)	0,00±0,00 (0)	24,29±4,24 (3.902.961)
Crash de coordenador	0,00±0,00 (1)	0,00±0,00 (0)	25,25±1,45 (4.008.616)
Partição de rede	0,05±0,05 (16.305)	0,05±0,10 (9.768)	19,01±1,74 (3.442.518)

Medida *dentro da janela de falta*, a partir das amostras de 10 s do YCSB, a taxa de erro sob ALL é de 56,7% no crash de réplica, 55,6% no de coordenador e 56,7% na partição (desvio padrão inferior a 0,1 ponto). Crash de réplica e partição incidem sobre o *mesmo* nó e coincidem na terceira casa (56,65% e 56,66%): o tipo de falta altera por quanto tempo as operações falham, e não quais. Os três ficam próximos dos 60% esperados de $RF = 3$ sobre cinco nós sob acesso uniforme; atribuímos a diferença à distribuição *zipfian* do *Workload A* [13]. Parte dos erros ocorre *após* a remoção: o serviço retorna ∼13–15 s após a remoção de um crash e ∼3 s após a remoção da partição. A taxa média (Tabela 2) e a instantânea são complementares: a primeira quantifica o custo amortizado ao longo da execução, e a segunda isola a severidade percebida enquanto o nó está indisponível.

Um resultado original está na comparação entre crash e partição sob ALL: a partição produz taxa de erro média *menor* (∼19%) que o crash (∼24–25%), embora a severidade instantânea medida no *mesmo* nó alvo seja idêntica (∼56,7%). A diferença não está, portanto, no que ocorre *durante* a falta, pois sob ALL um nó isolado porém vivo é tão inalcançável quanto um nó morto, e sim na *duração* do comprometimento do serviço: ∼98 s na partição contra ∼124–129 s nos crashes. Remover uma regra `iptables` restaura a conectividade de imediato, ao passo que um processo reiniciado precisa antes reingressar no *cluster*. Essa dissociação entre *remoção da falta* e *restauração do serviço* não havia sido caracterizada em trabalhos correlatos. A Figura 3 (centro) mostra ainda um breve pico de erro de QUORUM (∼18%) nos primeiros ∼10 s da partição, invisível nas métricas agregadas, que atribuímos à convergência do *gossip*.

6.2 Recuperabilidade: tempo de reparo

A Tabela 3 apresenta o MTTR por tipo de falta (RQ3). Os crashes se recuperam em ∼15–19 s e a partição em ∼32 s, o que atribuímos à reconvergência das visões do *gossip* dos dois lados. Esse MTTR de plano de controle difere da recuperação percebida pelo cliente: na partição o serviço retorna ∼3 s após a remoção, muito antes de o *gossip* convergir, enquanto nos crashes as duas grandezas praticamente coincidem (∼13–15 s). Modelos parametrizados por MTTR devem explicitar qual recuperação representa a indisponibilidade.

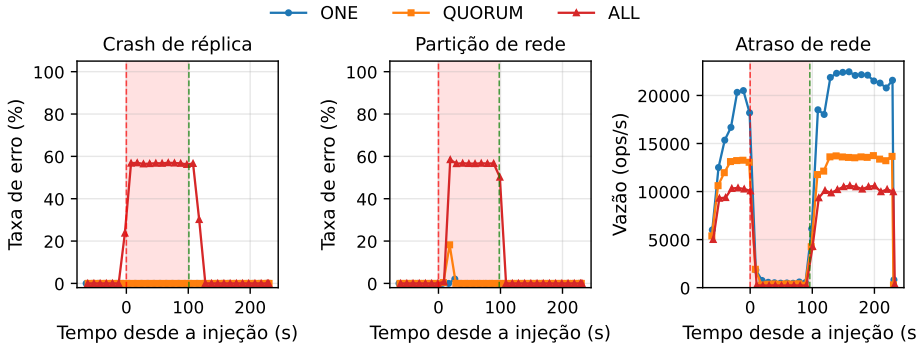


Figura 3. Séries temporais a 50 *threads* (repetição mediana), com o tempo realinhado à injeção da falta ($t = 0$) e a janela de falta sombreada. Esquerda e centro: taxa de erro sob crash de réplica e partição: apenas ALL reage, com ONE e QUORUM em ∼0%. Direita: vazão sob atraso de rede, os três níveis degradam conjuntamente.

Tabela 3. Tempo de reparo (s): da remoção da falta até `nodetool status` mostrar cinco nós ativos (plano de controle). Média ± desvio padrão; $n = 27$ por tipo de falta (3 níveis × 3 cargas × 3 repetições).

Tipo de falta	**Média ± DP**	**Mín.**	**Máx.**
Crash de réplica	15,19±2,76	11	25
Crash de coordenador	19,15±7,27	12	34
Partição de rede	31,93±7,90	23	47

6.3 Performability sob atraso e perda

A Tabela 4 apresenta a degradação de vazão sob faltas de temporização. A degradação é severa (∼97–98%) e *praticamente independente do nível de consistência*: as diferenças entre níveis são da ordem do desvio padrão (0,2–0,6 ponto). A taxa de erro permanece nula, tratando-se de falha de temporização, e não de omissão. Atribuímos a independência do CL ao caminho de escrita: toda escrita é enviada a todas as réplicas, de modo que um nó lento atrasa as operações que o envolvem, qualquer que seja o número de confirmações aguardadas.

O p99 da execução inteira (9–22 ms) esconde isso, pois as operações lentas são ∼1% do total. Por janela de 10 s, o p99 sobe de ∼11 ms para uma mediana de ∼920 ms durante a falta, e a perda é pior que o atraso (∼1,2–2,3 s contra ∼0,6–0,9 s) apesar de degradação de vazão igual.

6.4 Síntese

O nível de consistência controla o compromisso *sob faltas de disponibilidade* (RQ1), mas é *inócuo sob faltas de temporização* (RQ2): protege contra um nó indisponível, não contra degradação de rede.

Tabela 4. Degradação de vazão (%) sob faltas de temporização (média ± desvio padrão sobre cargas e repetições; $n = 9$ por célula).

Tipo de falta	ONE	QUORUM	ALL
Atraso de rede	97,54±0,45	97,60±0,44	98,35±0,61
Perda de pacotes	97,22±0,53	96,70±0,22	98,28±0,28

6.5 Ameaças à validade

Seis ameaças devem ser registradas. A primeira é de generalização: cada tipo de falta foi injetado em um *único nó alvo* fixo, escolha deliberada que isola o tipo de falta como único fator variável entre crash e partição, mas torna a severidade absoluta de ∼56,7% específica da faixa de *tokens* daquele nó e das chaves quentes do gerador *zipfian*. Dessa mesma dependência decorre a variância mais alta observada sob ALL em carga elevada. Segundo, três repetições por configuração ($n = 9$ por falta e nível, $n = 27$ por falta no MTTR) tornam as estimativas de dispersão imprecisas; as conclusões apoiam-se em contrastes muito maiores (zero contra 19–25% de erros; 97–98% de degradação em todos os níveis), e diferenças da ordem de um desvio padrão não são afirmadas. O MTTR é sondado via SSH, o que limita sua resolução a ∼10 s. Terceiro, os crashes são desligamentos ordenados. Uma verificação posterior em um novo cluster (30 *threads*, quatro repetições) encontrou zero operações com falha sob ONE e QUORUM também com `SIGKILL` e com queda de máquina emulada; mas sob ALL o serviço levou ∼48 s para retornar após `SIGKILL` (contra ∼18 s após parada ordenada), e a queda de máquina causou ∼20 s em que mais de 1% das operações levaram mais de 200 ms sob ONE e QUORUM. Nossos tempos de recuperação de crash são, portanto, limites inferiores para falhas abruptas. Quarto, a injeção em instante fixo não mede o MTTF, apenas o MTTR. Quinto, o cliente usou os cinco nós como pontos de contato sob balanceamento padrão; roteamento sensível à latência poderia atenuar a degradação sob faltas de temporização. Por fim, uma execução de partição apresentou comportamento anômalo por instabilidade transitória do cluster, tendo sido reexecutada após verificação de saúde.

7 Conclusão

Este trabalho mediu o impacto dos níveis de consistência do Cassandra sobre a disponibilidade, o desempenho e a recuperação do serviço sob injeção controlada de faltas. Em 135 execuções, crash e partição são mascarados por ONE e QUORUM, mas causam ∼20–25% de erro sob ALL, enquanto faltas de temporização degradam a vazão em ∼98% em todos os níveis. O MTTR vai de ∼15 s (crash) a ∼32 s (partição), e a restauração do serviço se dissocia da remoção da falta: ∼3 s após uma partição, mas só depois que o nó que caiu reingressa.

Este artigo relata a etapa experimental de um projeto de pesquisa de graduação em andamento. Como trabalho futuro, pretende-se aumentar as repetições por célula; injetar crashes abruptos e faltas múltiplas, concorrentes ou correlacionadas; investigar outras cargas e fatores de replicação; aplicar o método a outros

sistemas, como armazenamentos baseados em Raft (p. ex., etcd), para testar se os compromissos se generalizam; e construir um modelo analítico de dependabilidade (redes de Petri estocásticas ou diagramas de blocos de confiabilidade) parametrizado pelo MTTR aqui medido, o que permitirá validação cruzada entre disponibilidades medidas e previstas, além de análise de sensibilidade ao MTTF.

Referências

1. Lakshman, A., Malik, P.: Cassandra: a decentralized structured storage system. ACM SIGOPS Operating Systems Review **44**(2), 35–40 (2010)
2. Brewer, E.A.: Towards robust distributed systems. In: Proc. ACM PODC, pp. 7–10 (2000)
3. Gilbert, S., Lynch, N.: Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. ACM SIGACT News **33**(2), 51–59 (2002)
4. Avižienis, A., Laprie, J.C., Randell, B., Landwehr, C.: Basic concepts and taxonomy of dependable and secure computing. IEEE Trans. Dependable and Secure Computing **1**(1), 11–33 (2004)
5. Meyer, J.F.: On evaluating the performability of degradable computing systems. IEEE Trans. Computers **C-29**(8), 720–731 (1980). https://doi.org/10.1109/TC.1980.1675654
6. Ferreira, S., Andrade, E., Mendonça, J.: Uma abordagem experimental para avaliar os níveis de consistência do banco de dados NoSQL Cassandra. In: Anais do XXII Simpósio em Sistemas Computacionais de Alto Desempenho (WSCAD), pp. 156–167. SBC, Porto Alegre (2021). https://doi.org/10.5753/wscad.2021.18520
7. Borges, I., Andrade, E., Silva, F.A., Callou, G.: Availability evaluation of a video surveillance system with distributed storage. Cluster Computing (2025). https://doi.org/10.1007/s10586-024-04962-x
8. Silva, A., Callou, G.: Models for availability evaluation of file servers in private clouds. Computing (2025). https://doi.org/10.1007/s00607-024-01372-0
9. Gomes, C., Borba, E., Tavares, E., Nogueira de O. Junior, M.: Performability model for assessing NoSQL DBMS consistency. In: Proc. 2019 IEEE International Systems Conference (SysCon), pp. 1–6. IEEE (2019). https://doi.org/10.1109/SYSCON.2019.8836757
10. Rosselli, M., Niemann, R., Ivanov, T., Tolle, K., Zicari, R.V.: Benchmarking the availability and fault tolerance of Cassandra. In: Big Data Benchmarking (WBDB 2015). LNCS, vol. 10044, pp. 87–95. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-49748-8_5
11. Rahman, M.R., Golab, W., AuYoung, A., Keeton, K., Wylie, J.J.: Toward a principled framework for benchmarking consistency. In: Proc. 8th USENIX Workshop on Hot Topics in System Dependability (HotDep). USENIX Association (2012)
12. Kingsbury, K.: Jepsen: Cassandra. Relatório de análise Jepsen (2013). `https://aphyr.com/posts/294-jepsen-cassandra`, acesso em 17 set. 2026
13. Cooper, B.F., Silberstein, A., Tam, E., Ramakrishnan, R., Sears, R.: Benchmarking cloud serving systems with YCSB. In: Proc. ACM SoCC, pp. 143–154 (2010)
14. Cavalcanti, C.F., Callou, G.R.A.: Fault-injection scripts and data for "Empirical Dependability Measurement of Apache Cassandra under Fault Injection" (2026). `https://github.com/caaiofarias/cassandra-fault-injection-ladc2026`